Enterprise Multi-Engine Data Warehouse & BI System

Milestone 1: Architectural Design Document & Entity-Relationship Blueprint

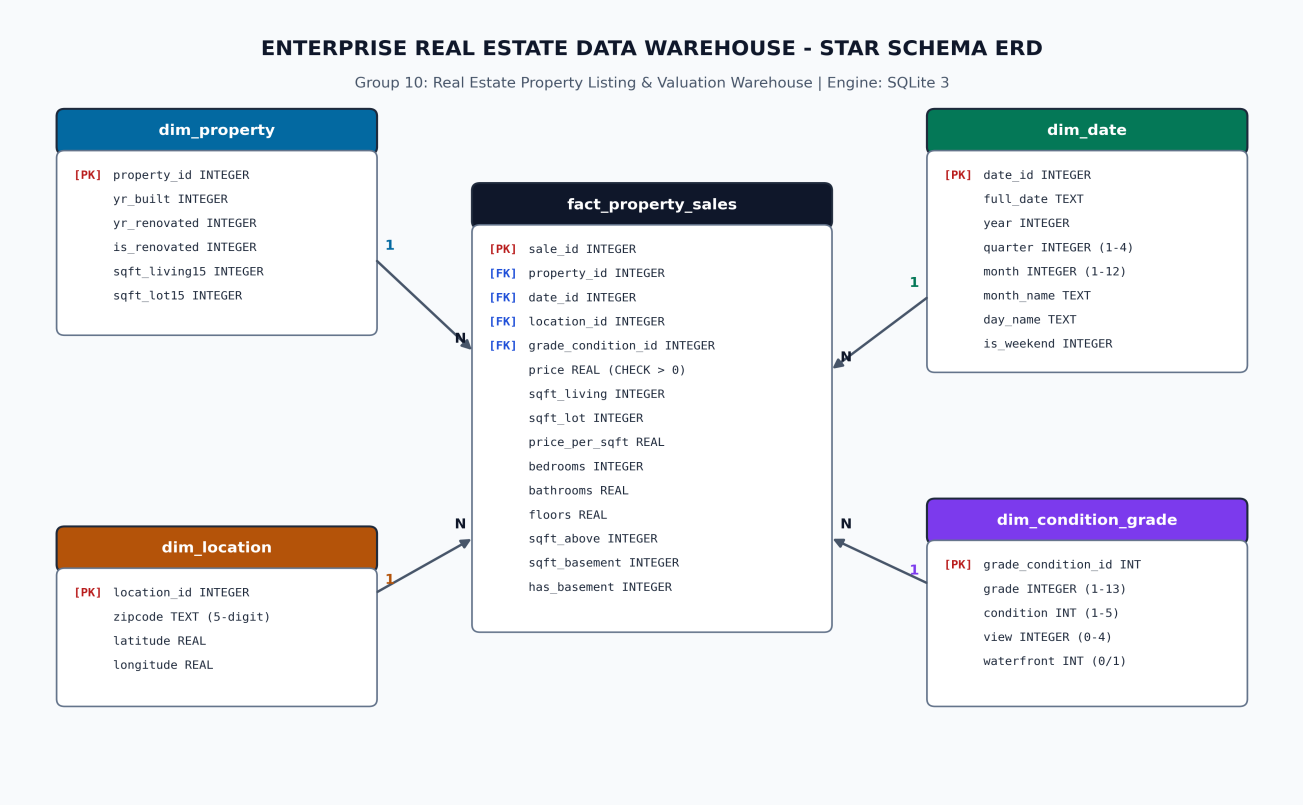
Domain:	Real Estate Property Listing & Valuation Warehouse (Group 10)
Course:	Data Warehousing (LEC/LAB) — Lesson 8 Final Project
Target Engines:	Relational SQLite (enterprise_warehouse.db) & Document MongoDB (real_estate_warehouse)
Instructor:	Prof. Rob Malitao
Due Date:	October 26, 2026 | Milestone 1 Deliverable

1. Executive Summary & Architectural Overview

To empower RetailPulse Global Inc. and executive leadership with rapid historical market intelligence without straining live transactional systems, this project implements a hybrid **Multi-Engine Data Warehouse Architecture**. The architecture unifies a structured, highly indexed relational **Star Schema (SQLite)** designed for sub-second aggregations and temporal OLAP slicing with a flexible, semi-structured **Document Store (MongoDB)** optimized for hierarchical architectural inspections, rich text reviews, and geospatial indexing.

2. Visual Star Schema Entity-Relationship Diagram (ERD)

The core relational warehouse is modeled in 3NF dimensional separation using a star schema. A central fact table (fact_property_sales) is surrounded by four specialized dimension tables maintaining strict 1-to-Many ($1:N$) referential integrity with foreign key constraints (ON DELETE RESTRICT).


ENTERPRISE REAL ESTATE DATA WAREHOUSE - STAR SCHEMA ERD
Group 10: Real Estate Property Listing & Valuation Warehouse | Engine: SQLite 3
dim_property
[PK] property_id INTEGER
yr_built INTEGER
yr_renovated INTEGER
is_renovated INTEGER
sqft_living15 INTEGER
sqft_lot15 INTEGER
dim_date
[PK] date_id INTEGER
full_date TEXT
year INTEGER
quarter INTEGER (1-4)
month INTEGER (1-12)
month_name TEXT
day_name TEXT
is_weekend INTEGER
fact_property_sales
[PK] sale_id INTEGER
[FK] property_id INTEGER
[FK] date_id INTEGER
[FK] location_id INTEGER
[FK] grade_condition_id INTEGER
price REAL (CHECK > 0)
sqft_living INTEGER
sqft_lot INTEGER
price_per_sqft REAL
bedrooms INTEGER
bathrooms REAL
floors REAL
sqft_above INTEGER
sqft_basement INTEGER
has_basement INTEGER
dim_location
[PK] location_id INTEGER
zipcode TEXT (5-digit)
latitude REAL
longitude REAL
dim_condition_grade
[PK] grade_condition_id INT
grade INTEGER (1-13)
condition INT (1-5)
view INTEGER (0-4)
waterfront INT (0/1)
1
N
1
N
N
1
N
1

3. Relational Schema Data Dictionary (SQLite: enterprise_warehouse.db)

Each table in the core SQLite relational database is governed by explicit DDL constraints, primary keys, referential integrity rules, and CHECK constraints preventing data anomalies from contaminating downstream marts.

Table: fact_property_sales (Central Transactional Fact)

Column Name	Type	Key / Constraint	Business Definition
sale_id	INTEGER	PRIMARY KEY AUTO	Unique surrogate transaction sales identifier.
property_id	INTEGER	FK (dim_property)	Refers to physical dwelling asset.
date_id	INTEGER	FK (dim_date)	Refers to transaction closing calendar date.
location_id	INTEGER	FK (dim_location)	Refers to geographic postal zone / centroid.
grade_condition_id	INTEGER	FK (dim_cond_grade)	Refers to structural assessment & vista score.
price	REAL	NOT NULL, CHECK > 0	Final recorded property sales price in USD.
sqft_living	INTEGER	NOT NULL, CHECK > 0	Interior finished dwelling area in sqft.
sqft_lot	INTEGER	NOT NULL, CHECK > 0	Total land parcel footprint in sqft.
price_per_sqft	REAL	NOT NULL, CHECK > 0	Unit valuation metric (price / sqft_living).
bedrooms	INTEGER	CHECK >= 0	Number of dedicated bedrooms.
bathrooms	REAL	CHECK >= 0	Total bathrooms (0.25 fractional increments).
floors	REAL	CHECK > 0	Total vertical finished dwelling stories.
has_basement	INTEGER	CHECK IN (0, 1)	Binary indicator flag (1=Basement, 0=None).

Dimension Tables Summary

Dimension	Primary Key	Key Attributes	Governance & Bounds
dim_property	property_id (PK)	yr_built, yr_renovated, is_renovated, sqft_living15, sqft_lot15	yr_built (1800-2050), sqft > 0
dim_location	location_id (PK AUTO)	zipcode, latitude, longitude	UNIQUE(zip, lat, long), lat/long bounds
dim_condition_grade	grade_condition_id (PK AUTO)	grade, condition, view, waterfront	grade (1-13), condition (1-5), view (0-4)
dim_date	date_id (PK YYYYMMDD)	full_date, year, quarter, month, day, is_weekend	quarter (1-4), month (1-12), day (1-31)

4. Semi-Structured MongoDB Document Store Architecture

MongoDB is integrated to store rich nested architectural specifications and qualitative customer/inspector feedback. Strict database-level governance is enforced using $jsonSchema with validationLevel='strict' and validationAction='error'.

Collection 1: sales_documents

Stores nested property specifications, geospatial coordinates, and verified closing sales data:

```
{ "$jsonSchema": { "bsonType": "object", "required": ["property_id", "sale_date", "price", "sqft_living",
"architectural_specs", "location"], "properties": { "property_id": {"bsonType": ["int", "long"]},
"sale_date": {"bsonType": "string", "pattern": "^\\d{4}-\\d{2}-\\d{2}$"}, "price": {"bsonType": ["double",
"int", "long"], "minimum": 1.0}, "sqft_living": {"bsonType": ["int", "long"], "minimum": 1},
"architectural_specs": { "bsonType": "object", "required": ["bedrooms", "bathrooms", "floors", "grade",
"condition"], "properties": { "bedrooms": {"bsonType": "int", "minimum": 0}, "bathrooms": {"bsonType":
["double", "int"], "minimum": 0.0}, "grade": {"bsonType": "int", "minimum": 1, "maximum": 13}, "condition":
{"bsonType": "int", "minimum": 1, "maximum": 5} } }, "location": { "bsonType": "object", "required":
["zipcode", "coordinates"], "properties": { "zipcode": {"bsonType": "string", "pattern": "^\\d{5}$"},
"coordinates": {"bsonType": "array", "minItems": 2, "maxItems": 2} } } } } }
```

Collection 2: feedback_documents

Captures semi-structured property surveys, licensed inspections, and stakeholder sentiment:

```
{ "$jsonSchema": { "bsonType": "object", "required": ["review_id", "property_id", "review_date", "rating",
"reviewer_category", "inspection_passed"], "properties": { "review_id": {"bsonType": "string"},
"property_id": {"bsonType": ["int", "long"]}, "review_date": {"bsonType": "string", "pattern":
"^\\d{4}-\\d{2}-\\d{2}$"}, "rating": {"bsonType": ["int", "long"], "minimum": 1, "maximum": 5},
"reviewer_category": {"bsonType": "string", "enum": ["Buyer", "Seller", "Licensed Inspector", "Certified
Appraiser", "Tenant"]}, "inspection_passed": {"bsonType": "bool"}, "sentiment_score": {"bsonType": ["double",
"int"]}, "comments": {"bsonType": "string"}, "architectural_highlights": {"bsonType": "array", "items":
{"bsonType": "string"}} } } }
```

5. ETL Pipeline Ingestion & Data Quality Audit Verification

The automated Python pipeline (capstone_etl.py) cleanses operational anomalies extracted from the landing zone (./landing_zone/) and populates both storage engines while enforcing governance. Below are the verified execution metrics recorded during the Milestone 1 audit execution:

Pipeline Execution Metric	Value Recorded	Verification Status
Raw Sales Records Ingested from Landing Zone	5,720 rows	VERIFIED (100% Extracted)
Exact Duplicate Rows Dropped	150 rows dropped	DEDUPLICATED
Constraint Violators & Corrupt Keys Quarantined	70 rows dropped	GOVERNED
Clean Records Loaded to SQLite (fact_property_sales)	5,500 rows loaded	SUCCESSFUL COMMIT
Clean Documents Loaded to MongoDB (sales_documents)	5,500 documents	SUCCESSFUL COMMIT
Clean Feedback Docs Loaded (feedback_documents)	1,189 documents	SUCCESSFUL COMMIT
SQLite CHECK & FK Constraint Verification Suite	All tests triggered & passed	PASSED (Integrity Confirmed)
MongoDB Strict $jsonSchema Validation Suite	All violations blocked	PASSED (WriteError Blocked)